Next.js 技術ブログ 環境構築マニュアル（Week 1）

初学者向けに、Week 1 でやったことを再現できるようにまとめたマニュアルです。
このマニュアルを上から順番に実行すると、同じサイトが完成します。

全体の流れ

```
Step 1：Node.jsのバージョン確認
Step 2：プロジェクトの作成
Step 3：ライブラリのインストール
Step 4：フォルダとファイルの作成
Step 5：lib/articles.ts を書く（データ層）
Step 6：最初の記事（MDX）を書く
Step 7：記事一覧ページを作る
Step 8：記事詳細ページを作る
Step 9：ホームページを作る
Step 10：スタイルを設定する
Step 11：GitHubにpushする
Step 12：Vercelにデプロイする
```

Step 1：Node.jsのバージョン確認

ターミナルを開いて以下を打ちます。

```bash
node -v
npm -v
```

Node.js v18 以上、npm v9 以上であれば問題ありません。`command not found` が出た場合は https://nodejs.org からインストールしてください。

Step 2：プロジェクトの作成

作業フォルダを作り、Next.js プロジェクトを生成します。

```bash
mkdir ~/Projects
cd ~/Projects
npx create-next-app@latest my-site --typescript --tailwind --app --eslint --no-src-d
```

途中でいくつか質問が出ます。すべてEnterキーを押してデフォルトのまま進めてください。`React Compiler を使うか？`と聞かれたら`No`を選びます。

完了したらプロジェクトを起動します。

```bash
cd my-site
npm run dev
```

ブラウザで`http://localhost:3000`を開いてNext.jsのデフォルト画面が出れば成功です。

> **コマンドの意味**
>
> - `npx`：npmに登録されているツールを一時的に実行する
> - `create-next-app`：Next.js公式のプロジェクト生成ツール
> - `--typescript`：TypeScriptを使う設定
> - `--tailwind`：Tailwind CSSを使う設定
> - `--app`：App Router（最新構成）を使う設定
> - `npm run dev`：開発用サーバーを起動する

Step 3：ライブラリのインストール

別のターミナルタブを開いて以下を打ちます（`npm run dev`は動かしたまま）。

```bash
cd ~/Projects/my-site
npm install gray-matter next-mdx-remote rehype-highlight remark-gfm
```

> **ライブラリの意味**
>
> - `gray-matter`：MDXのフロントマター（記事のメタ情報）をJavaScriptオブジェクトに変換する
> - `next-mdx-remote`：MDXの本文をReactコンポーネントとして画面に描画する
> - `rehype-highlight`：コードブロックにシンタックスハイライト（色付け）を適用する
> - `remark-gfm`：Markdownの拡張記法（テーブルなど）を有効にする

`npm warn EBADENGINE`という警告が出ることがありますが、エラーではないので無視して大丈夫です。

Step 4：フォルダとファイルの作成

VSCode でプロジェクトを開きます。

```bash
code .
```

ターミナルで以下を打って必要なフォルダとファイルを作ります。

```bash
mkdir content
mkdir content/articles
mkdir lib
touch lib/articles.ts
touch content/articles/python-sequential-files.mdx
mkdir -p app/articles
touch app/articles/page.tsx
mkdir -p "app/articles/[slug]"
touch "app/articles/[slug]/page.tsx"
```

完了後のフォルダ構成はこうなります。

```
my-site/
├── app/
|   ├── articles/
|   |   ├── page.tsx            ← 記事一覧ページ
|   |   └── [slug]/
|   |       └── page.tsx        ← 記事詳細ページ
|   ├── globals.css
|   ├── layout.tsx
|   └── page.tsx                ← ホームページ
├── content/
|   └── articles/
|       └── python-sequential-files.mdx  ← 最初の記事
├── lib/
|   └── articles.ts             ← データ層（最重要）
└── public/
    └── downloads/              ← .pyファイルを置く場所（後で作る）
```

Step 5：lib/articles.ts を書く

`lib/articles.ts` はこのサイト全体のデータ層です。 MDXファイルを読み込んでページに渡す「窓口」の役割を持ちます。

`lib/articles.ts` を開いて以下を貼り付けてください。

```typescript
```

```
import fs from 'fs'
import path from 'path'
import matter from 'gray-matter'

const ARTICLES_DIR = path.join(process.cwd(), 'content/articles')

export type ArticleMeta = {
  title: string
  date: string
  category: string
  tags: string[]
  excerpt: string
  slug: string
  draft?: boolean
}

export type Article = ArticleMeta & {
  content: string
}

export function getAllArticles(): ArticleMeta[] {
  const files = fs.readdirSync(ARTICLES_DIR)
  return files
    .filter(f => f.endsWith('.mdx'))
    .map(filename => {
      const raw = fs.readFileSync(path.join(ARTICLES_DIR, filename), 'utf-8')
      const { data } = matter(raw)
      return {
        ...(data as ArticleMeta),
        slug: filename.replace('.mdx', ''),
        date: data.date instanceof Date
          ? data.date.toISOString().split('T')[0]
          : String(data.date),
      }
    })
    .filter(article => !article.draft)
    .sort((a, b) => (a.date > b.date ? -1 : 1))
}

export function getArticleBySlug(slug: string): Article {
  const filepath = path.join(ARTICLES_DIR, `${slug}.mdx`)
  const raw = fs.readFileSync(filepath, 'utf-8')
  const { data, content } = matter(raw)
  return {
    ...(data as ArticleMeta),
    slug,
```

```
    content,
    date: data.date instanceof Date
      ? data.date.toISOString().split('T')[0]
      : String(data.date),
  }
}

export function getAllTags(): string[] {
  const articles = getAllArticles()
  const tags = articles.flatMap(a => a.tags ?? [])
  return [...new Set(tags)]
}
```

> **各関数の役割**
>
> - `getAllArticles()`：全記事のメタ情報一覧を返す → 記事一覧ページで使う
> - `getArticleBySlug()`：slug指定で1記事のデータ＋本文を返す → 記事詳細ページで使う
> - `getAllTags()`：全記事のタグを重複なしで返す → タグページで使う

Step 6：最初の記事（MDX）を書く

`content/articles/python-sequential-files.mdx` を開いて以下を貼り付けます。

```markdown
```

```
---
title: Pythonで連番ファイルを生成する方法
date: 2026-03-14
category: Python
tags: [automation, beginner, snippet]
excerpt: 研究や開発で大量のファイルを一括生成したいときに使えるPythonスクリプトを解説します。
draft: false
---

## はじめに

研究で実験データを整理するとき、`data_001.txt` から `data_100.txt` まで連番のファイルを
一括で作りたい場面はよくあります。手作業でやると時間がかかるうえにミスも起きます。
Pythonなら10行以内で解決できます。

## コード
```python
import os

def generate_files(prefix, start, end, ext, output_dir="."):
    os.makedirs(output_dir, exist_ok=True)
    for i in range(start, end + 1):
        filename = f"{prefix}_{i:03d}.{ext}"
        filepath = os.path.join(output_dir, filename)
        with open(filepath, "w") as f:
            f.write(f"# {filename}\n")
    print(f"{end - start + 1}個のファイルを生成しました")

generate_files("data", 1, 10, "txt", "output")
```

コードの解説

`i:03d` は「3桁のゼロ埋め」という意味です。`1` を渡すと `001` になります。
`os.makedirs` は出力先フォルダが存在しない場合に自動で作成します。
`exist_ok=True` をつけることで、フォルダが既に存在してもエラーにならないようにしています。

使い方

`prefix` にファイル名の接頭辞、`start` と `end` に開始・終了番号、`ext` に拡張子を渡します。
```

> **フロントマターのルール**
>
> - `---` で囲まれた部分がフロントマター（記事のメタ情報）
> - `title`：記事タイトル
> - `date`：投稿日（YYYY-MM-DD形式）

> - `category`：カテゴリ（1記事につき1つ）
> - `tags`：タグ（複数付けてOK）
> - `excerpt`：一覧ページに表示する要約文
> - `draft: true` にすると公開されない

Step 7：記事一覧ページを作る

`app/articles/page.tsx` を開いて以下を貼り付けます。

typescript

```typescript
import Link from 'next/link'
import { getAllArticles } from '@/lib/articles'

export default function ArticlesPage() {
  const articles = getAllArticles()

  return (
    <main className="max-w-2xl mx-auto px-4 py-12">
      <h1 className="text-3xl font-bold mb-8">記事一覧</h1>
      <ul className="space-y-6">
        {articles.map(article => (
          <li key={article.slug} className="border border-gray-200 rounded-lg p-6">
            <Link href={`/articles/${article.slug}`}>
              <h2 className="text-xl font-semibold hover:underline">
                {article.title}
              </h2>
            </Link>
            <p className="text-sm text-gray-500 mt-1">{article.date} · {article.cate
            <p className="text-gray-700 mt-2">{article.excerpt}</p>
            <div className="flex gap-2 mt-3">
              {article.tags.map(tag => (
                <span key={tag} className="text-xs bg-gray-100 text-gray-600 px-2 py
                  {tag}
                </span>
              ))}
            </div>
          </li>
        ))}
      </ul>
    </main>
  )
}
```

ブラウザで `http://localhost:3000/articles` を開いて記事一覧が表示されれば成功です。

Step 8：記事詳細ページを作る

`app/articles/[slug]/page.tsx` を開いて以下を貼り付けます。

```typescript
```

```
import { getAllArticles, getArticleBySlug } from '@/lib/articles'
import { MDXRemote } from 'next-mdx-remote/rsc'
import remarkGfm from 'remark-gfm'

export async function generateStaticParams() {
  const articles = getAllArticles()
  return articles.map(article => ({ slug: article.slug }))
}

export default async function ArticlePage({
  params,
}: {
  params: Promise<{ slug: string }>
}) {
  const { slug } = await params
  const article = getArticleBySlug(slug)

  return (
    <main className="max-w-2xl mx-auto px-4 py-12">
      <div className="mb-8">
        <p className="text-sm text-gray-500 mb-2">
          {article.date} · {article.category}
        </p>
        <h1 className="text-3xl font-bold mb-4">{article.title}</h1>
        <div className="flex gap-2">
          {article.tags.map(tag => (
            <span
              key={tag}
              className="text-xs bg-gray-100 text-gray-600 px-2 py-1 rounded"
            >
              {tag}
            </span>
          ))}
        </div>
      </div>
      <article className="prose max-w-none">
        <MDXRemote
          source={article.content}
          options={{
            mdxOptions: {
              remarkPlugins: [remarkGfm],
            },
          }}
        />
      </article>
    </main>
```

```
  )
}
```

> **`generateStaticParams` の役割** Next.js に「このURLパターンで存在するページ一覧」を事前に教える関数です。 これによってビルド時に全記事ページが静的HTMLとして生成されます。

Step 9：ホームページを作る

`app/page.tsx` を開いて全体を以下に差し替えます。

```typescript
```

```
import Link from 'next/link'
import { getAllArticles } from '@/lib/articles'

export default function Home() {
  const articles = getAllArticles().slice(0, 3)

  return (
    <main className="max-w-2xl mx-auto px-4 py-12">
      <section className="mb-16">
        <h1 className="text-4xl font-bold mb-4">
          研究・開発効率化ナレッジベース
        </h1>
        <p className="text-gray-600 text-lg leading-relaxed">
          Python自動化・環境構築・情報系理論・研究効率化に関する技術記事とツールをまとめています。
          コードはすべてコピー・ダウンロードして使えます。
        </p>
        <div className="flex gap-4 mt-6">
          <Link
            href="/articles"
            className="bg-black text-white px-5 py-2 rounded-lg hover:bg-gray-800 tr
          >
            記事を読む
          </Link>
          <Link
            href="/about"
            className="border border-gray-300 px-5 py-2 rounded-lg hover:bg-gray-50
          >
            このサイトについて
          </Link>
        </div>
      </section>

      <section>
        <div className="flex items-center justify-between mb-6">
          <h2 className="text-2xl font-bold">最新記事</h2>
          <Link href="/articles" className="text-sm text-gray-500 hover:underline">
            すべて見る →
          </Link>
        </div>
        <ul className="space-y-4">
          {articles.map(article => (
            <li key={article.slug} className="border border-gray-200 rounded-lg p-5"
              <Link href={`/articles/${article.slug}`}>
                <h3 className="font-semibold hover:underline">{article.title}</h3>
              </Link>
              <p className="text-sm text-gray-500 mt-1">
```

```
                {article.date} · {article.category}
              </p>
              <p className="text-gray-600 text-sm mt-2">{article.excerpt}</p>
            </li>
          ))}
        </ul>
      </section>
    </main>
  )
}
```

Step 10：スタイルを設定する

`app/globals.css` を開いて全体を以下に差し替えます。

```
css
```

```
@import "tailwindcss";

@layer base {
  .prose h2 {
    font-size: 1.5rem;
    font-weight: 700;
    margin-top: 2rem;
    margin-bottom: 0.75rem;
  }
  .prose h3 {
    font-size: 1.25rem;
    font-weight: 600;
    margin-top: 1.5rem;
    margin-bottom: 0.5rem;
  }
  .prose p {
    margin-bottom: 1rem;
    line-height: 1.75;
  }
  .prose pre {
    background: #1e1e1e;
    color: #d4d4d4;
    padding: 1rem;
    border-radius: 0.5rem;
    overflow-x: auto;
    margin: 1rem 0;
  }
  .prose code {
    font-family: monospace;
    font-size: 0.9rem;
  }
  .prose :not(pre) > code {
    background: #f3f4f6;
    padding: 0.1rem 0.3rem;
    border-radius: 0.25rem;
    font-size: 0.85rem;
  }
}
```

Step 11：GitHubにpushする

まずGitHubで新しいリポジトリを作成します。 https://github.com を開いて右上の `+` → `New repository` をクリックします。

設定は以下にします。

- Repository name：`my-site`
- `Add a README file` はチェックしない

`Create repository` を押した後、ターミナルで以下を打ちます。`YOUR_USERNAME` は自分のGitHubユーザー名に変えてください。

```bash
cd ~/Projects/my-site
git init
git add .
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/YOUR_USERNAME/my-site.git
git push -u origin main
```

GitHubのページをリロードしてファイルが表示されれば成功です。

> **コマンドの意味**
>
> - `git init`：このフォルダをGitで管理し始める
> - `git add .`：全ファイルをコミット対象として選択する
> - `git commit -m "..."`：選択したファイルの状態をスナップショットとして保存する
> - `git branch -M main`：ブランチ名をmainに変更する
> - `git remote add origin ...`：ローカルフォルダをGitHubリポジトリと紐付ける
> - `git push -u origin main`：GitHubにアップロードする

Step 12：Vercelにデプロイする

https://vercel.com をブラウザで開いて、GitHubアカウントでログインします。

1. `Add New...` → `Project` をクリック
2. `my-site` の横の `Import` をクリック
3. 設定は何も変えずに `Deploy` をクリック

1〜2分でデプロイが完了します。`Domains` に表示された `xxx.vercel.app` のURLが公開URLです。

今後の記事追加手順

Week 1 完了後、新しい記事を追加するときの手順はこれだけです。

```bash
# 1. 新しいMDXファイルを作る
touch content/articles/新しい記事のslug.mdx

# 2. フロントマターと本文を書く（Step 6 の形式に従う）

# 3. GitHubにpushする（Vercelが自動でデプロイしてくれる）
git add .
git commit -m "add: 新しい記事のタイトル"
git push
```

以上でインターネット上に記事が公開されます。

トラブルシューティング

`Objects are not valid as a React child` エラーが出た `gray-matter` が日付をDateオブジェクトとして読み込むために起きます。 `lib/articles.ts` の `date` の部分に以下の変換処理が入っているか確認してください。

```typescript
date: data.date instanceof Date
  ? data.date.toISOString().split('T')[0]
  : String(data.date),
```

`remote origin already exists` エラーが出た URLを間違えた場合は以下で修正できます。

```bash
git remote set-url origin https://github.com/YOUR_USERNAME/my-site.git
git push -u origin main
```

VSCodeで `@plugin` に警告が出る Tailwind v4 の新しい構文をVSCodeのCSS拡張が認識できていないだけです。 動作には影響しないので無視して大丈夫です。